

Q1. Merge Sort (Question 1 and 2)

Time Complexity : $O(n \log n)$

Space Complexity : $O(n)$

Merge Sort recursively divides the array into two halves until single element remains.

$\therefore TC \rightarrow O(n \log n)$.

Additional temporary array is required during merging, $\therefore SC \rightarrow O(n)$.

Quick Sort (Question 1 and 2)

Time Complexity : $O(n \log n)$ - Avg case
 $O(n^2)$ - Worst case

Space Complexity : $O(\log n)$

It partitions the array around a pivot and recursively sorts the subarrays. On avg, the partitions are balanced, giving $O(n \log n)$ and in worst case (already sorted array), the complexity becomes $O(n^2)$.

Optimized Approach - Choosing a random pivot significantly reduces the chance of the worst case.

Heap Sort (Question 1 and 2)

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$

It first builds a heap in $O(n)$ time and then repeatedly removes the maximum element, each taking $O(\log n)$, making the time complexity $O(n \log n)$.

Since, the sorting is done in place, only constant extra memory is required.

Optimized Approach - It is already an efficient in-place sorting algorithm.

T1 Q1

```
#include <iostream>
```

```
using namespace std;
```

```
void merge(int arr[], int l, int m, int r) {
```

```
int n1 = m - l + 1;
```

```
int n2 = r - m;
```

```
int L[n1], R[n2];
```

```
for(int i = 0; i < n1; i++)
```

```
L[i] = arr[l + i];
```

```
for(int j = 0; j < n2; j++)
```

```
R[j] = arr[m + 1 + j];
```

```
int i = 0, j = 0, k = l;
```

```
while(i < n1 && j < n2) {
```

```
if(L[i] <= R[j])
```

```
arr[k++] = L[i++];
```

```
else
```

```
arr[k++] = R[j++];
```

```
}
```

```
while(i < n1)
```

```
arr[k++] = L[i++];
```

```
while(j < n2)
arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int l, int r) {
if(l < r) {
int m = (l + r) / 2;
mergeSort(arr, l, m);
mergeSort(arr, m + 1, r);
merge(arr, l, m, r);
}
}
```

```
int main() {
int n;
cin >> n;
```

```
int arr[n];
```

```
for(int i = 0; i < n; i++)
cin >> arr[i];
```

```
mergeSort(arr, 0, n - 1);
```

```
for(int i = 0; i < n; i++)
cout << arr[i] << " ";
```

```
return 0;
```



```
}
```

T1 Q2

```
#include <iostream>
```

```
using namespace std;
```

```
void merge(int arr[], int l, int m, int r){
```

```
int n1=m-l+1,n2=r-m;
```

```
int L[n1],R[n2];
```

```
for(int i=0;i<n1;i++) L[i]=arr[l+i];
```

```
for(int i=0;i<n2;i++) R[i]=arr[m+1+i];
```

```
int i=0,j=0,k=l;
```

```
while(i<n1 && j<n2){
```

```
if(L[i]<=R[j]) arr[k++]=L[i++];
```

```
else arr[k++]=R[j++];
```

```
}
```

```
while(i<n1) arr[k++]=L[i++];
```

```
while(j<n2) arr[k++]=R[j++];
```

```
}
```

```
void mergeSort(int arr[],int l,int r){
```

```
if(l<r){
```

```
int m=(l+r)/2;
```

```
mergeSort(arr,l,m);
```

```
mergeSort(arr,m+1,r);
```

```
cout<<"Orders Above Median: "<<count<<endl;
```

```
cout<<"Difference: "<<arr[n-1]-arr[0];
```

```
return 0;
```

```
}
```

T2 Q1

```
#include <iostream>
```

```
using namespace std;
```

```
int partition(int arr[], int low, int high){
```

```
int pivot=arr[high];
```

```
int i=low-1;
```

```
for(int j=low;j<high;j++){
```

```
if(arr[j]<pivot){
```

```
    i++;
```

```
    swap(arr[i],arr[j]);
```

```
}
```

```
}
```

```
swap(arr[i+1],arr[high]);
```

```
return i+1;
```

```
}
```

```
void quickSort(int arr[],int low,int high){
```

```
if(low<high){
```

```
int pi=partition(arr,low,high);
```

```
merge(arr,l,m,r);  
}  
}
```

```
int main(){  
int n;  
cin>>n;
```

```
int arr[n];
```

```
for(int i=0;i<n;i++)  
cin>>arr[i];
```

```
mergeSort(arr,0,n-1);
```

```
for(int i=0;i<n;i++)  
cout<<arr[i]<<" ";
```

```
cout<<endl;
```

```
int median=arr[n/2];  
cout<<"Median: "<<median<<endl;
```

```
int count=0;  
for(int i=0;i<n;i++)  
if(arr[i]>median)  
count++;
```

```
for(int j=low;j<high;j++){  
    if(arr[j]>pivot){  
        i++;  
        swap(arr[i],arr[j]);  
    }  
}
```

```
swap(arr[i+1],arr[high]);  
return i+1;  
}
```

```
void quickSort(int arr[],int low,int high){  
    if(low<high){  
        int pi=partition(arr,low,high);  
        quickSort(arr,low,pi-1);  
        quickSort(arr,pi+1,high);  
    }  
}
```

```
int main(){  
    int n;  
    cin>>n;  
  
    int arr[n];  
    long long sum=0;  
  
    for(int i=0;i<n;i++){  
        cin>>arr[i];
```



```
quickSort(arr,low,pi-1);
quickSort(arr,pi+1,high);
}
}
```

```
int main(){
int n;
cin>>n;

int arr[n];

for(int i=0;i<n;i++)
cin>>arr[i];
```

```
quickSort(arr,0,n-1);

for(int i=0;i<n;i++)
cout<<arr[i]<<" ";
```

```
return 0;
}
```

T2 Q2

```
#include <iostream>
using namespace std;
```

```
int partition(int arr[],int low,int high){
int pivot=arr[high];
int i=low-1;
```

```
sum+=arr[i];
```

```
}
```

```
quickSort(arr,0,n-1);
```

```
for(int i=0;i<n;i++)
```

```
cout<<arr[i]<<" ";
```

```
cout<<endl;
```

```
cout<<"Top 5: ";
```

```
long long topSum=0;
```

```
for(int i=0;i<5;i++){
```

```
cout<<arr[i]<<" ";
```

```
topSum+=arr[i];
```

```
}
```

```
cout<<endl;
```

```
cout<<"Average of Top 5: "<<topSum/5.0<<endl;
```

```
double avg=(double)sum/n;
```

```
int count=0;
```

```
for(int i=0;i<n;i++)
```

```
if(arr[i]>avg)
```

```
count++;
```

```

for(int i=n-1;i>0;i--){
    swap(arr[0],arr[i]);
    heapify(arr,i,0);
}
}

```

```

int main(){
    int n;
    cin>>n;

```

```

    int arr[n];

```

```

    for(int i=0;i<n;i++){
        cin>>arr[i];
    }

```

```

    heapSort(arr,n);

```

```

    for(int i=0;i<n;i++){
        cout<<arr[i]<<" ";
    }

```

```

    return 0;
}

```

T3 Q2

```

#include <iostream>
using namespace std;

```

```

void heapify(int arr[],int n,int i){
    int largest=i;

```

```
cout<<"Values Above Overall Average: "<<count;
```

```
return 0;
```

```
}
```

T3 Q1

```
#include <iostream>
```

```
using namespace std;
```

```
void heapify(int arr[],int n,int i){
```

```
int largest=i;
```

```
int l=2*i+1;
```

```
int r=2*i+2;
```

```
if(l<n && arr[l]>arr[largest])
```

```
largest=l;
```

```
if(r<n && arr[r]>arr[largest])
```

```
largest=r;
```

```
if(largest!=i){
```

```
swap(arr[i],arr[largest]);
```

```
heapify(arr,n,largest);
```

```
}
```

```
}
```

```
void heapSort(int arr[],int n){
```

```
for(int i=n/2-1;i>=0;i--)
```

```
heapify(arr,n,i);
```

```
int arr[n];
double sum=0;

for(int i=0;i<n;i++){
    cin>>arr[i];
    sum+=arr[i];
}

heapSort(arr,n);

for(int i=0;i<n;i++)
    cout<<arr[i]<<" ";

cout<<endl;

cout<<"Fastest: "<<arr[0]<<endl;
cout<<"Slowest: "<<arr[n-1]<<endl;

double avg=sum/n;
cout<<"Average: "<<avg<<endl;

int count=0;
for(int i=0;i<n;i++)
    if(arr[i]<avg)
        count++;

cout<<"Cases Faster Than Average: "<<count<<endl;
```



```
int l=2*i+1;
```

```
int r=2*i+2;
```

```
if(l<n && arr[l]>arr[largest])
```

```
largest=l;
```

```
if(r<n && arr[r]>arr[largest])
```

```
largest=r;
```

```
if(largest!=i){
```

```
swap(arr[i],arr[largest]);
```

```
heapify(arr,n,largest);
```

```
}
```

```
}
```

```
void heapSort(int arr[],int n){
```

```
for(int i=n/2-1;i>=0;i--)
```

```
heapify(arr,n,i);
```

```
for(int i=n-1;i>0;i--){
```

```
swap(arr[0],arr[i]);
```

```
heapify(arr,i,0);
```

```
}
```

```
}
```

```
int main(){
```

```
int n;
```

```
cin>>n;
```

```
cout<<"Percentage: "<<(count*100.0)/n<<"%";
```

```
return 0;
```

```
}
```

```

1  #include <iostream>
2  using namespace std;
3
4  void merge(int arr[], int l, int m, int r) {
5      int n1 = m - l + 1;
6      int n2 = r - m;
7      int L[n1], R[n2];
8      for(int i = 0; i < n1; i++)
9          L[i] = arr[l + i];
10     for(int j = 0; j < n2; j++)
11         R[j] = arr[m + 1 + j];
12     int i = 0, j = 0, k = l;
13     while(i < n1 && j < n2) {
14         if(L[i] <= R[j])
15             arr[k++] = L[i++];
16         else
17             arr[k++] = R[j++];
18     }
19     while(i < n1)
20         arr[k++] = L[i++];
21     while(j < n2)
22         arr[k++] = R[j++];
23 }
24 void mergeSort(int arr[], int l, int r) {
25     if(l < r) {
26         int m = (l + r) / 2;
27         mergeSort(arr, l, m);
28         mergeSort(arr, m + 1, r);

```



```

6
450
1200
800
650
300
1500
300 450 650 800 1200 1500

```

```

1 #include <iostream>
2 using namespace std;
3 void merge(int arr[], int l, int m, int r){
4     int n1=m-l+1,n2=r-m;
5     int L[n1],R[n2];
6     for(int i=0;i<n1;i++) L[i]=arr[l+i];
7     for(int i=0;i<n2;i++) R[i]=arr[m+1+i];
8     int i=0,j=0,k=1;
9     while(i<n1 && j<n2){
10         if(L[i]<=R[j]) arr[k++]=L[i++];
11         else arr[k++]=R[j++];
12     }
13     while(i<n1) arr[k++]=L[i++];
14     while(j<n2) arr[k++]=R[j++];
15 }
16 void mergeSort(int arr[],int l,int r){
17     if(l<r){
18         int m=(l+r)/2;
19         mergeSort(arr,l,m);
20         mergeSort(arr,m+1,r);
21         merge(arr,l,m,r);
22     }
23 }
24 int main(){
25     int n;
26     cin>>n;
27     int arr[n];

```



```

4
12
25
22
40
12 22 25 40
Median: 25
Orders Above Median: 1
Difference: 28

```



```

1  #include <iostream>
2  using namespace std;
3  int partition(int arr[], int low, int high){
4      int pivot=arr[high];
5      int i=low-1;
6      for(int j=low;j<high;j++){
7          if(arr[j]<pivot){
8              i++;
9              swap(arr[i],arr[j]);
10         }
11     }
12     swap(arr[i+1],arr[high]);
13     return i+1;
14 }
15 void quickSort(int arr[],int low,int high){
16     if(low<high){
17         int pi=partition(arr,low,high);
18         quickSort(arr,low,pi-1);
19         quickSort(arr,pi+1,high);
20     }
21 }
22 int main(){
23     int n;
24     cin>>n;
25     int arr[n];
26     for(int i=0;i<n;i++)
27         cin>>arr[i];

```

```

6
120
45
78
200
60
95
45 60 78 95 120 200

```

```

1  #include <iostream>
2  using namespace std;
3  int partition(int arr[],int low,int high){
4      int pivot=arr[high];
5      int i=low-1;
6      for(int j=low;j<high;j++){
7          if(arr[j]>pivot){
8              i++;
9              swap(arr[i],arr[j]);
10         }
11     }
12     swap(arr[i+1],arr[high]);
13     return i+1;
14 }
15 void quickSort(int arr[],int low,int high){
16     if(low<high){
17         int pi=partition(arr,low,high);
18         quickSort(arr,low,pi-1);
19         quickSort(arr,pi+1,high);
20     }
21 }
22 int main(){
23     int n;
24     cin>>n;
25     int arr[n];
26     long long sum=0;
27     for(int i=0;i<n;i++){

```



```

5
500
1200
7700
2500
1500
7700 2500 1500 1200 500
Top 5: 7700 2500 1500 1200 500
Average of Top 5: 2680
Values Above Overall Average: 1

```



```

1  #include <iostream>
2  using namespace std;
3  void heapify(int arr[],int n,int i){
4      int largest=i;
5      int l=2*i+1;
6      int r=2*i+2;
7      if(l<n && arr[l]>arr[largest])
8          largest=l;
9      if(r<n && arr[r]>arr[largest])
10         largest=r;
11     if(largest!=i){
12         swap(arr[i],arr[largest]);
13         heapify(arr,n,largest);
14     }
15 }
16 void heapSort(int arr[],int n){
17     for(int i=n/2-1;i>=0;i--)
18         heapify(arr,n,i);
19     for(int i=n-1;i>0;i--){
20         swap(arr[0],arr[i]);
21         heapify(arr,i,0);
22     }
23 }
24 int main(){
25     int n;
26     cin>>n;
27     int arr[n];
28     for(int i=0;i<n;i++)

```







```

6
850
1200
950
600
1500
1000
600 850 950 1000 1200 1500

```

```

1  #include <iostream>
2  using namespace std;
3  void heapify(int arr[],int n,int i){
4      int largest=i;
5      int l=2*i+1;
6      int r=2*i+2;
7      if(l<n && arr[l]>arr[largest])
8          largest=l;
9      if(r<n && arr[r]>arr[largest])
10         largest=r;
11     if(largest!=i){
12         swap(arr[i],arr[largest]);
13         heapify(arr,n,largest);
14     }
15 }
16 void heapSort(int arr[],int n){
17     for(int i=n/2-1;i>=0;i--)
18         heapify(arr,n,i);
19     for(int i=n-1;i>0;i--){
20         swap(arr[0],arr[i]);
21         heapify(arr,i,0);
22     }
23 }
24 int main(){
25     int n;
26     cin>>n;
27     int arr[n];
28     double sum=0;

```



```

8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.5%

```